ACICP302ICS

Module 1 Practical Lab Assessment Brief

Source-Code-Guided OT Simulation Lab

Document detail	Information
Module	Module 1 Integrated ICS Network Security & Attacks: Analyze, Test & Recover the Supplied OT Simulation
Activity title	Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation
Assessment workflow	Governed Practical Laboratories evidence workflow
Maximum score / pass	100 marks / 50% practical pass threshold
Contribution category	Practical Laboratories
Submission mode	Private learner GitHub repository + associated evidence

Read this first

This is a source-code-guided practical. Students use and analyze the supplied loopback-safe simulation; they are not required to build a replacement simulator. Use only the canonical student source and follow the safety boundary in this document.

International Cybersecurity and Digital Forensics Academy (ICDFA) | Version 2.0 | 05 October 2026

1. Scenario

You are an OT security analysis team assigned a small simulated process based on the course-provided Modbus/TCP source. Your task is to inspect and run the supplied loopback-safe simulation in an isolated Ubuntu virtual machine, establish normal operation, analyze protocol and guard behavior, complete controlled demonstrations, assess limitations and verify recovery. You are not required to build a replacement simulator or guard.

The source is supplied; the analysis is yours

Use the canonical course source at `labs/ot-security/student-lab-source/`. The archived original at `labs/ot-security/archive-original/` contains unsafe network defaults and must not be executed. Submit your own observations, captures, risk analysis, report and evidence not the course repository as if it were your project.

2. Objectives

- Safely locate and verify the canonical course source, record its commit and run it only inside the assigned Ubuntu VM on loopback.
- Explain the supplied architecture, process model, Modbus/TCP data points and direct/guarded paths using source and runtime evidence.
- Capture normal loopback traffic, identify a request/response and connect packet fields to process state.
- Document the local service exposure and process-related risks within the authorized scope.
- Analyze the supplied guard's checks, events, assumptions and limitations without misrepresenting capabilities.
- Plan and run at least three approved bounded demonstrations, compare expected and observed results, and verify recovery.
- Submit a private evidence repository with traceable, learner-created artifacts and defend conclusions accurately.

3. Full student briefing

Before beginning, obtain access to the private course repository from the instructor or use the instructor-provided archive. Read the Student Lab Guide. Prepare a fresh, instructor-approved Ubuntu guest VM and clean snapshot. Do not use bridged networking or port forwarding. Complete approved repository/package downloads during temporary setup access, then disconnect the VM's network adapter before running the simulation.

Run only `labs/ot-security/student-lab-source/`. It is the canonical course copy configured to use `127.0.0.1`. Never execute files in `labs/ot-security/archive-original/`; the archived bundle uses a non-loopback destination (`192.168.2.139`) and broad listener binds (`0.0.0.0`). If the loopback check fails or scope is unclear, stop and contact the instructor. Do not edit the code to force a pass.

Complete Activities 00–08 in order unless the instructor explicitly waives an activity: preflight and safety; source architecture and normal baseline; supplied Modbus/TCP data model; loopback packet analysis; local exposure mapping; supplied guard analysis; controlled validation using at least three approved demonstration scripts; recovery/evidence/report/defence; and the full source-code lab run. Use supplied script defaults unchanged and run one demonstration at a time. Availability/load demonstrations require explicit instructor approval.

Submit your own work through the governed Practical Laboratories workflow: a private learner/team GitHub evidence repository and associated evidence links, with matching commit/branch/file references. Do not submit the course source repository as your own. Cite its URL and exact commit; clearly separate instructor-provided source from your analysis. Maximum marks are protected after the first submission. Follow the opening/due dates shown in the LMS (WAT).

4. Course source and access

Course repository: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks

Canonical student source: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks/tree/main/labs/ot-security/student-lab-source

Complete Ubuntu VM guide: https://github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks/blob/main/labs/ot-security/STUDENT_LAB_GUIDE.md

Private repository access

The course repository is private. The instructor/course owner must grant access or provide an approved archive through the LMS. Do not make the course repository public solely to avoid access setup. Learner evidence repositories should remain private unless the instructor specifies otherwise.

5. Required activities and evidence

Activity 00 - Lab preflight and safety

Prepare the assigned Ubuntu VM and verify the exact course source and authorization boundary before starting any simulation process.

Expected tasks: Obtain access to the private course repository from the instructor, or use the approved archive. Read the Student Lab Guide. Create/restore the instructor-assigned clean VM snapshot. Confirm the adapter is not Bridged and port forwarding is absent; disconnect the adapter after approved downloads and dependency setup. Locate `labs/ot-security/student-lab-source/`. Do not execute files in `labs/ot-security/archive-original/`. Verify the student code uses `127.0.0.1` for clients and service binds. Stop and notify the instructor if the check fails; do not edit the source to make it pass. Record the course source commit, VM/snapshot, date and WAT time, authorized scope, stop condition and recovery method. Begin the evidence log and private learner evidence workspace.

Evidence: Completed preflight/evidence log; source path and commit; loopback check; VM/snapshot details; topology draft; signed safety declaration.

Activity 01 - Architecture and plant baseline

Understand the supplied simulation architecture and establish a learner-observed normal baseline; do not build a replacement plant.

Expected tasks: Read `docs/reference-architecture.md` and inspect the files in `student-lab-source/plant_sim/`. Trace the simulated plant, Modbus/TCP service, direct dashboard, guard, guarded dashboard and supplied client scripts. Annotate the architecture with roles and data flow. Follow the guide to start only normal services. Observe the process and dashboards before any demonstration. Record normal values, update behavior and any differences between direct and guarded views; stop services as directed.

Evidence: Annotated topology/data-flow diagram; source references; screenshots of normal dashboards; baseline observations and evidence-log entry.

Activity 02 - Modbus/TCP data model and normal operations

Document the data model implemented by the supplied simulation and relate its definitions to observed normal behavior.

Expected tasks: Inspect the source definitions for process values, register addresses, units/scales, normal bounds, status/alarm semantics and update behavior. Create a data-point table. Mark unknowns as unknown; do not guess. Run the normal local simulation and verify selected reads using only the guide's documented local path. Explain the client/server roles and a normal request/response, linking values to the dashboard. Note any source/comment/display mismatch.

Evidence: Learner-prepared register map; source references; normal-read/dashboard evidence; concise transaction explanation.

Activity 03 - Packet visibility and protocol analysis

Capture normal Modbus/TCP traffic generated by the supplied simulation and explain what a matched request/response demonstrates.

Expected tasks: Use the Ubuntu guest loopback interface (`lo`) and documented Modbus/TCP port only. Start the normal services and an authorized read client as described in the guide; do not run abnormal demonstrations during this capture. Save a short, labeled PCAP/PCAPNG from your own VM. Identify a request and matching response in Wireshark. Explain the observed endpoint/port, transaction/protocol identifiers, function code, address/value and result; correlate the value with the register map and dashboard. State what the capture proves and what it does not prove, then stop capture and processes.

Evidence: Learner-created PCAP/PCAPNG; annotated request/response screenshots; analysis tied to baseline; capture-method evidence-log entry.

Activity 04 - Safe discovery and exposure mapping

Map only the local services present in the supplied source and assess their process relevance without probing external systems.

Expected tasks: Inspect source bind/target configuration and identify only documented loopback endpoints. Prepare a scope card naming VM, loopback target, permitted method, time window, expected state, stop condition and recovery action. Use source inspection and guide-approved read-only checks to verify services; do not widen ranges or use a non-loopback target. Record component role, protocol, process significance and potential consequence of exposure/misuse. Add evidence-based risk-register findings and preventative/detective controls.

Evidence: Approved scope card; local service inventory; relevant command/output or screenshot; risk register; short evidence-based conclusion.

Activity 05 - Analyze the supplied defensive OT guard

Explain the guard already included in the course source, its observed decisions, audit evidence and limitations; a new guard implementation is not required.

Expected tasks: Inspect `guard.py` and the guarded dashboard in the canonical student source. Create a decision table: each check, input, condition, decision/action and related event/log evidence. Confirm the interpretation from a normal run and approved scenario. Map demonstrated checks to risk-register findings. Discuss limits such as client authentication/authorization, direct-path access, availability, correlated values, threshold tuning and recovery. Do not change the source or thresholds unless the instructor assigns an explicit extension.

Evidence: Guard architecture/decision table; relevant source references; normal-pass/event sample; limitation analysis; updated risk entries.

Activity 06 - Controlled security validation

Compare direct and guarded behavior using approved bounded demonstrations supplied with the course source.

Expected tasks: Complete the test plan before execution: source/script, loopback path, baseline, expected outcome, evidence, stop condition and recovery. Select at least three scenarios permitted by the Student Lab Guide/instructor and use supplied parameters unchanged. Run one scenario at a time against the documented direct or guarded path. Never alter target, port, counts, duration or thread settings. Observe script output, dashboard/process state and relevant guard events; compare expected and actual behavior. Verify recovery after each scenario and report variable or unexpected results honestly.

Evidence: Approved test plan; evidence for at least three scenarios; expected-versus-observed comparison; recovery proof for each.

Activity 07 - Recovery, evidence and project defence

Complete cleanup and produce a defensible record of your work using the supplied simulation.

Expected tasks: Stop all services/demonstrations, verify the lab ports are no longer listening, and restore the approved state/snapshot as directed. Complete the evidence log, risk register, test comparison, final report and evidence index using course templates. Cite the course repository and exact source commit; distinguish supplied code from your analysis and any authorized modifications. Prepare a live defence showing normal state, one

approved scenario, observed guard response, limitations and verified recovery. Submit a private learner/team evidence repository URL plus matching branch/commit/file references through E-Campus.

Evidence: Final report; repository/commit reference; evidence index/log; risk register; comparison and recovery record; defence plan.

Activity 08 - Supplied-source Modbus/TCP OT security lab

Complete the Ubuntu VM hands-on run of the course source. This is the core practical, unless explicitly waived by the instructor.

Expected tasks: Follow every setup, isolation, dependency, run and cleanup step in the Student Lab Guide. Use only `labs/ot-security/student-lab-source/` and record its commit. The archived copy is not runnable student material. Verify loopback-only configuration, start plant, guard and dashboards, and record a baseline. Complete normal packet analysis and at least three instructor-approved demonstrations one at a time using supplied defaults. Compare direct and guarded results, explain the range/step/drift checks evidenced, describe limitations, and verify cleanup/recovery.

Evidence: Source/version record; topology/register map; baseline and comparison evidence; packet analysis; test plan/results; relevant event evidence; risk entry; limitations and cleanup statement.

6. Minimum acceptance and safety criteria

Criterion	Minimum standard
Source and scope	Correct source path/commit recorded; loopback check passes; no archive execution, bridged access or external target.
Architecture and baseline	Supplied roles and data flow accurately explained; baseline supported by learner observations.
Data model and packet analysis	Register/data-point map traceable to source; normal request/response accurately interpreted.
Exposure and risk	Local service inventory and process risks are evidence-based and within scope.
Guard analysis	Implemented checks and event evidence accurately described; limitations distinguished from recommendations.
Validation	At least three instructor-approved bounded scenarios; expected/observed results, evidence and recovery recorded.
Submission	Private learner evidence repository, final report, evidence log/index and matching commit/file references.

Stop conditions

Stop immediately if any target/bind is not `127.0.0.1`, a real/external system appears, the VM becomes unstable, the approved scope is unclear, or the result is unexpected. Notify the instructor and follow the recovery plan.

7. Evidence items for the LMS

Evidence item	What to submit	Marks mapping
1. Source, scope and architecture	Course source URL/commit; preflight; authorized scope; topology; baseline.	20
2. Data model and normal operation	Register/data-point map; normal dashboard/read evidence; transaction explanation.	15
3. Packet and exposure analysis	Loopback PCAP/PCAPNG; annotated request/response; local inventory; risk register.	15

Evidence item	What to submit	Marks mapping
4. Supplied guard analysis	Decision table/source references; event evidence; limitations/residual risk.	20
5. Validation and recovery	Approved test plan; at least three scenario results; comparison; recovery proof.	15
6. Report and defence	Final report, evidence log/index, private evidence-repo reference and oral/live defence.	15

8. Marking rubric - 100 marks

Criterion	Full-credit standard	Marks
Safe source use, architecture analysis and baseline	Canonical code used safely; correct architecture/data flow; clear normal baseline and reproducible evidence.	20
Modbus/TCP data model and normal-operation analysis	Accurate source-backed register map, roles, normal reads and explanation.	15
Packet visibility, protocol analysis and local exposure	Usable capture; accurate request/response interpretation; scoped service/risk analysis.	15
Supplied guard analysis and audit evidence	Correct explanation of implemented checks/events, assumptions and limitations.	20
Controlled validation, findings and recovery	At least three approved tests; evidence-based comparison; verified safe cleanup/recovery.	15
Professional report, evidence and defence	Traceable learner-created evidence; clear attribution; complete report and confident defence.	15
TOTAL	Practical pass threshold: 50%, subject to institutional standard.	100

9. Submission and cleanup checklist

- Use the assigned LMS dates and WAT times; submit before the due time.
- Create a private learner/team repository for your report and permitted evidence; submit the exact repository and matching commit/branch/file URLs.
- Do not commit credentials, tokens, private keys, real site data, or unapproved captures/logs. Do not upload another learner's evidence.
- Stop all lab processes, confirm local listeners have stopped, retain evidence only as authorized and restore the assigned clean snapshot if directed.
- Maximum marks are protected after the first submission. Review the complete package and all links before that submission.

Solution below:



ACICP302ICS — Module 1 Practical Lab: Final Report

Learner:	Sidra Shaikh
Student ID	C11/26/ACIS/17349
Date	10–11 October 2026
Evidence repository	https://github.com/SidraShaikh-2/ICDFA-Assignments/tree/main/ACICP302%20%E2%80%94%20ICS%20Network%20Security%20%26%20Attacks

Analyze, Test & Defend the Supplied Modbus/TCP OT Simulation

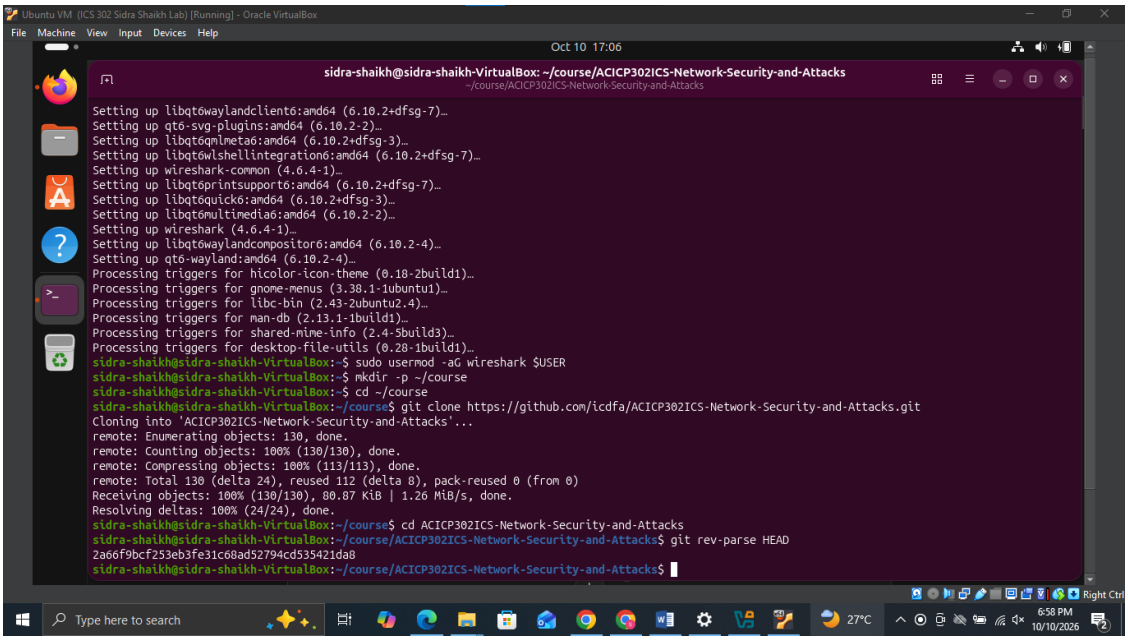
A short note on timekeeping before I begin: the clock inside my lab virtual machine ran one hour ahead of West Africa Time, so wherever I quote a time from a screenshot I am quoting the VM's own clock. To convert any of these to WAT, subtract one hour.

1. Source version, scope and safety

Before touching anything, I made sure I knew exactly which code I was dealing with and that it could only ever talk to my own machine. The instructor-supplied source lives in the public course repository at `github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks`, and I used only the canonical copy under `labs/ot-security/student-lab-source/`. I recorded the exact commit I worked from — `2a66f9bcf253eb3fe31c68ad52794cd535421da8`, authored on 5 October 2026 — so that everything in this report is traceable to one fixed version of the code. I did not run anything from the sibling `archive-original/` directory, which the brief warns contains unsafe non-loopback defaults.

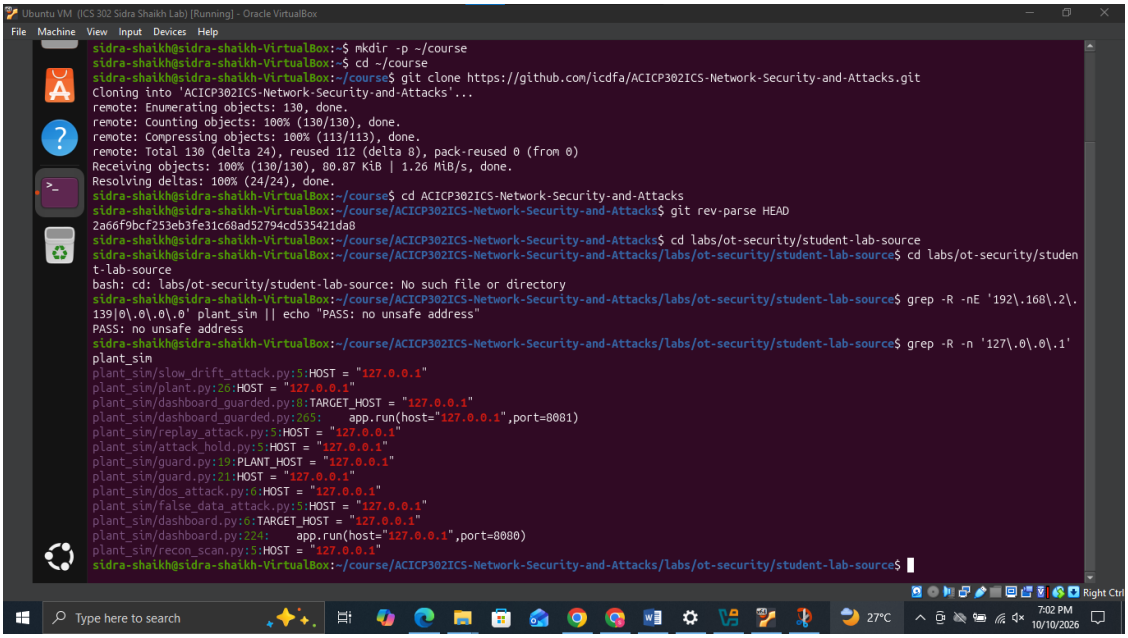
The lab ran inside a fresh Oracle VirtualBox guest running Ubuntu, with Python 3 and a clean snapshot named `clean-ready`. I only allowed the VM onto the network (through NAT) long enough to clone the repository and install the two dependencies, and I confirmed there were no port-forwarding rules at all. Once the downloads were finished I disconnected the network adapter entirely. Running `ip -br addr` afterwards confirmed the isolation: the only interface that was up was the loopback interface `lo` on `127.0.0.1`, while the ethernet adapter `enp0s3` showed as DOWN.

I then ran the loopback safety check the guide asks for, as a pure read-only inspection rather than editing anything to force a pass. Searching the source for the forbidden addresses (`grep -R -nE '192\.168\.2\.139|0\.0\.0\.0' plant_sim`) returned nothing, and searching for `127.0.0.1` showed that every client target and every service bind uses the loopback address. On that basis I treated the scope as confirmed: target `127.0.0.1` only, ports 5020, 5021, 8080 and 8081, and the supplied demonstration scripts with their parameters left exactly as written. My stop condition throughout was simple, if any target or bind had turned out not to be loopback, or an external system had appeared, or the VM had become unstable, I would have stopped and contacted the instructor. My recovery plan was to stop every process, confirm with `ss -ltn` that nothing was still listening, and restore the `clean-ready` snapshot.



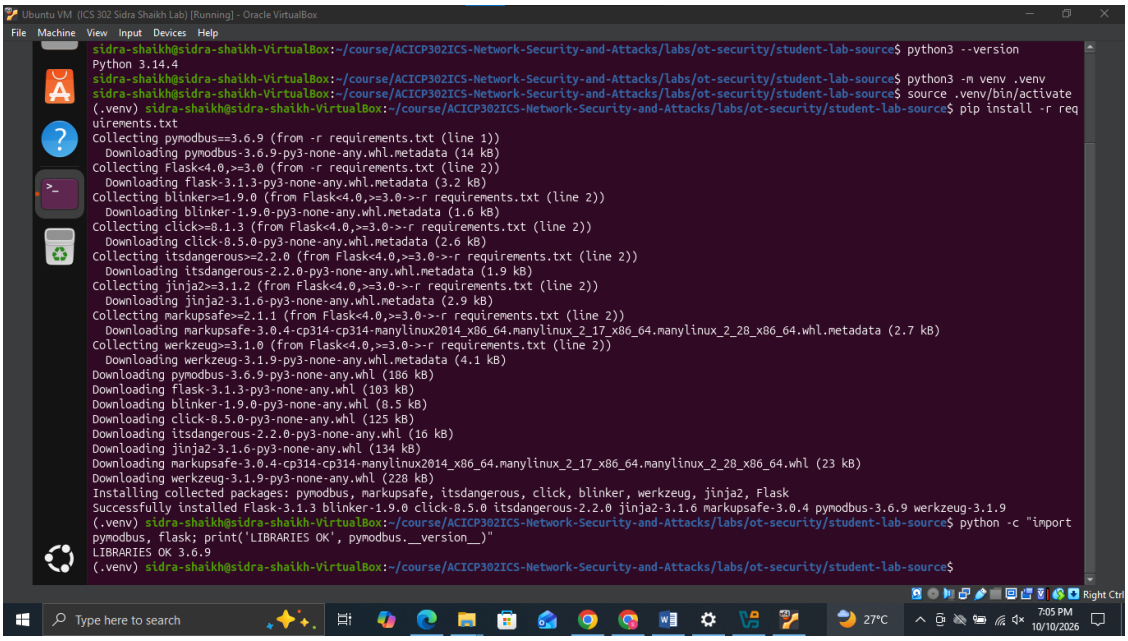
Source commit 2a66f9bc… recorded with git rev-parse HEAD

Figure: Source commit 2a66f9bc… recorded with git rev-parse HEAD



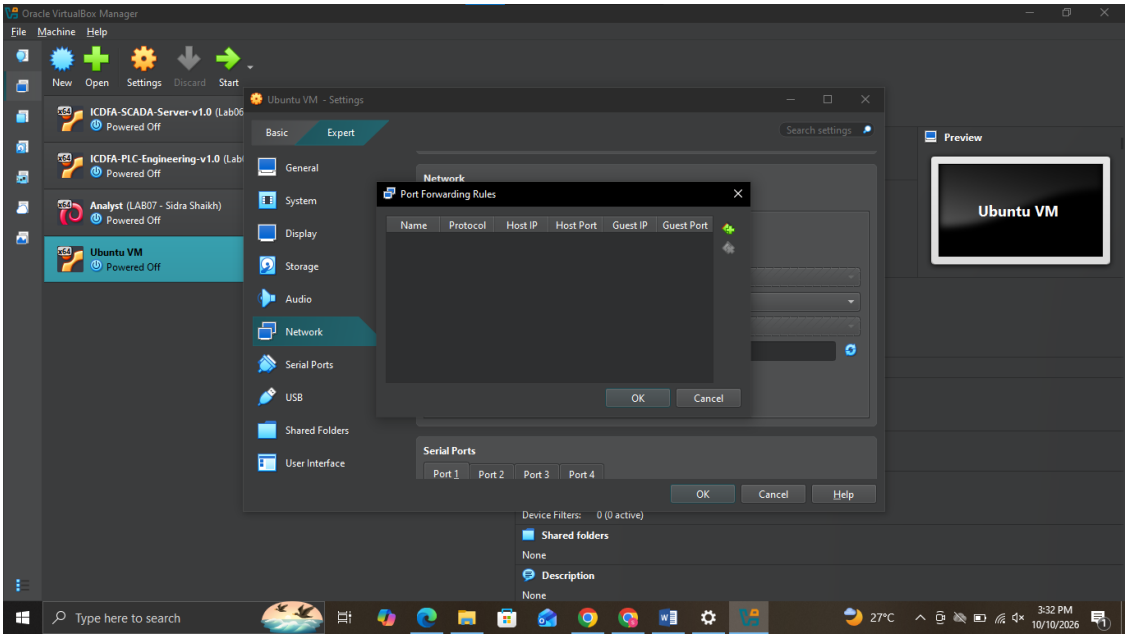
Loopback check — PASS, no unsafe address; all binds 127.0.0.1

Figure: Loopback check — PASS, no unsafe address; all binds 127.0.0.1



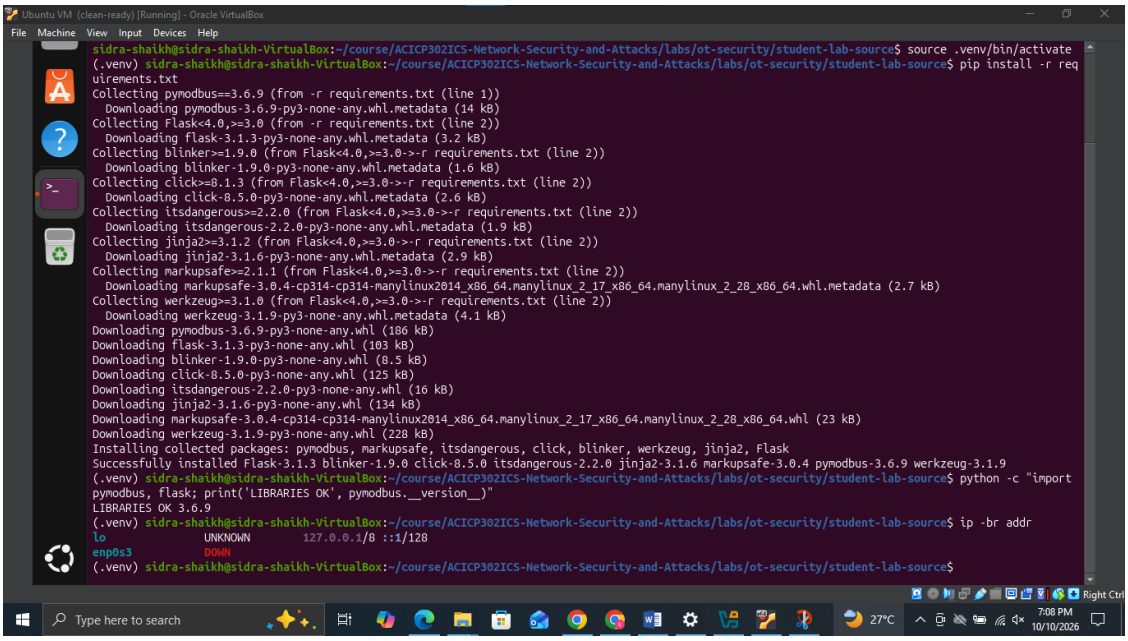
Dependencies installed — LIBRARIES OK 3.6.9

Figure: Dependencies installed — LIBRARIES OK 3.6.9



VirtualBox port-forwarding list empty

Figure: VirtualBox port-forwarding list empty



Isolated after setup — snapshot clean-ready; enp0s3 DOWN, only lo up

Figure: Isolated after setup — snapshot clean-ready; enp0s3 DOWN, only lo up

2. Architecture and plant baseline

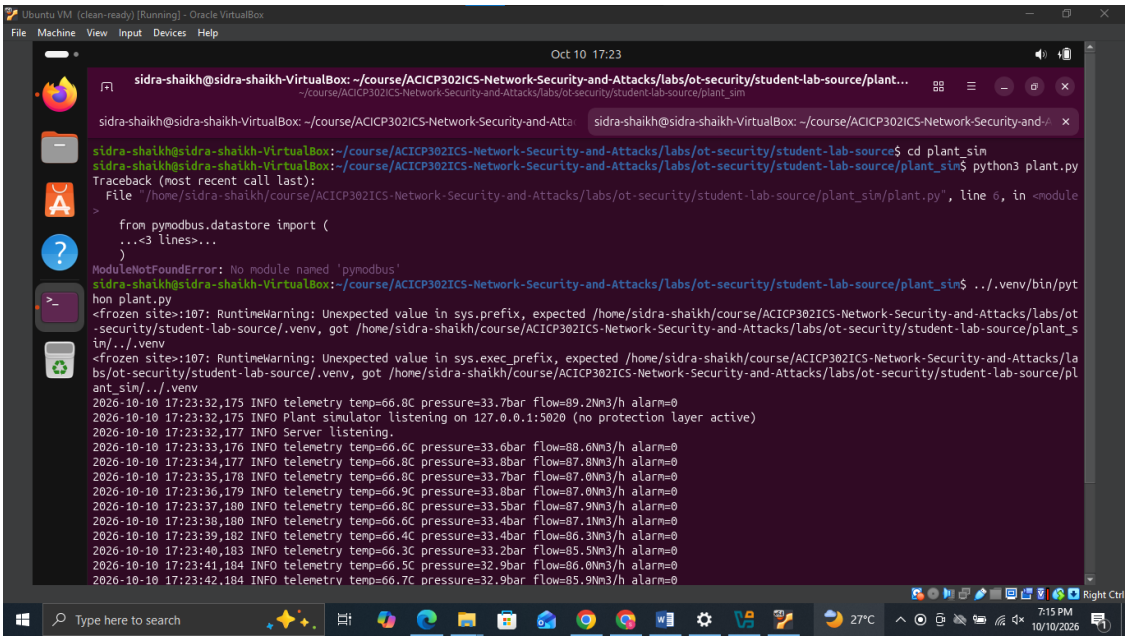
The simulation models a small hydrogen facility, and once I had it running I could see it is really four cooperating programs. At the centre is the plant itself (`plant.py`), a Modbus/TCP server on `127.0.0.1:5020` whose own startup log cheerfully announces that it is running with "no protection layer active". Sitting in front of it is the guard (`guard.py`) on `127.0.0.1:5021`, which inspects every write and only forwards the ones it considers safe down to the plant. Finally there are two web dashboards: the "undefended" one (`dashboard.py`) on port 8080, which reads the plant directly, and the "guarded" one (`dashboard_guarded.py`) on port 8081, which reads through the guard and also displays the guard's event log.

What this architecture creates is two different ways to reach the same plant. The direct path takes a client straight to port 5020 with nothing in the way, while the guarded path routes a client through the guard on 5021, which validates the request before anything reaches the plant. That distinction turns out to be the whole story of the lab.

Component	Source file	Serves on	Talks to	Role
Plant simulator	`plant_sim/plant.py`	`127.0.0.1:5020`	—	Simulated process, no protection
Defensive guard	`plant_sim/guard.py`	`127.0.0.1:5021`	forwards accepted writes to `:5020`	Rule-based write firewall
Direct dashboard	`plant_sim/dashboard.py`	`http://127.0.0.1:8080`	polls plant `:5020`	"UNDEFENDED" view
Guarded dashboard	`plant_sim/dashboard_guarded.py`	`http://127.0.0.1:8081`	polls guard `:5021`	"GUARDED" view + event log

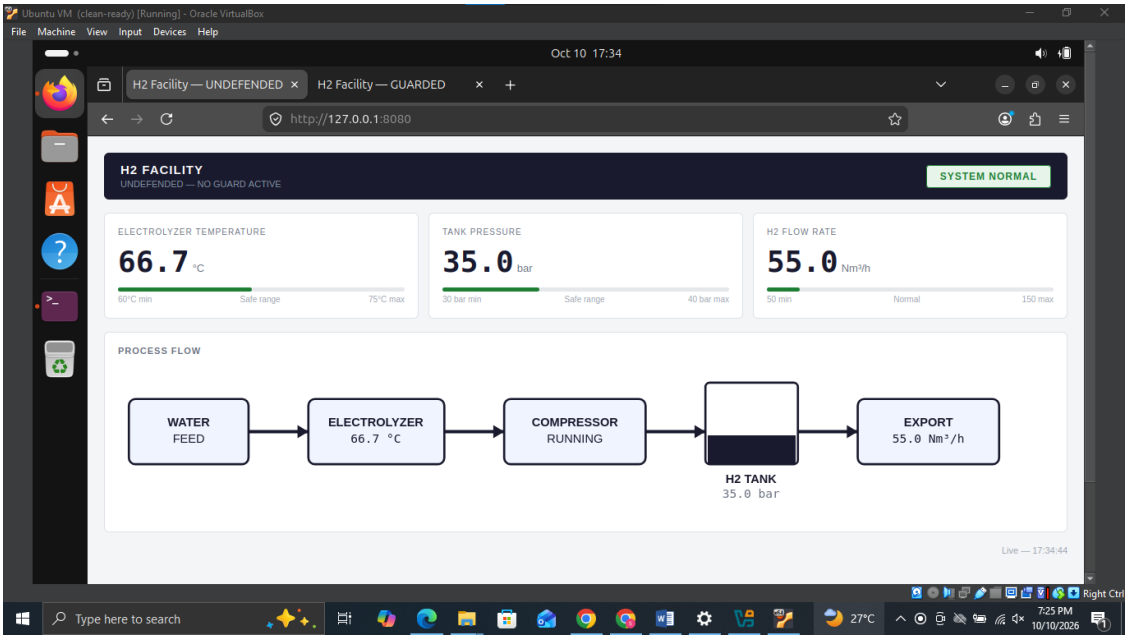
I started the four services in order and watched them for a while before doing anything else, so that I would know what "normal" actually looks like. The temperature hovered between roughly 66 and 71 °C, the pressure drifted around the low-to-high thirties of a bar, and the flow rate moved over the widest range

of the three, somewhere between about 53 and 89 Nm³/h. All of this gentle wandering stayed inside the safe bands, and both dashboards reported a healthy state — “SYSTEM NORMAL” on 8080 and “PROTECTED” on 8081. Crucially, the two views agreed with each other to within a few tenths, the tiny difference being the one-second gap between when each dashboard polls. Throughout this quiet period the guard’s detection log simply read “Waiting for attacks…”, which is exactly the clean-baseline evidence I wanted before provoking anything.



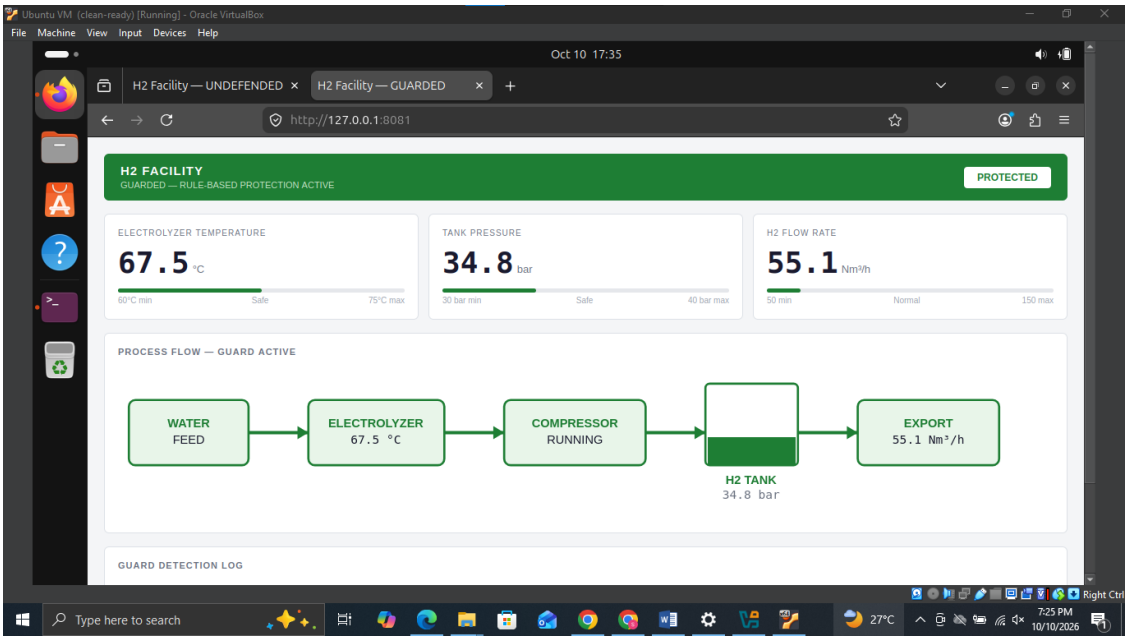
Plant listening on 127.0.0.1:5020 with per-second telemetry

Figure: Plant listening on 127.0.0.1:5020 with per-second telemetry



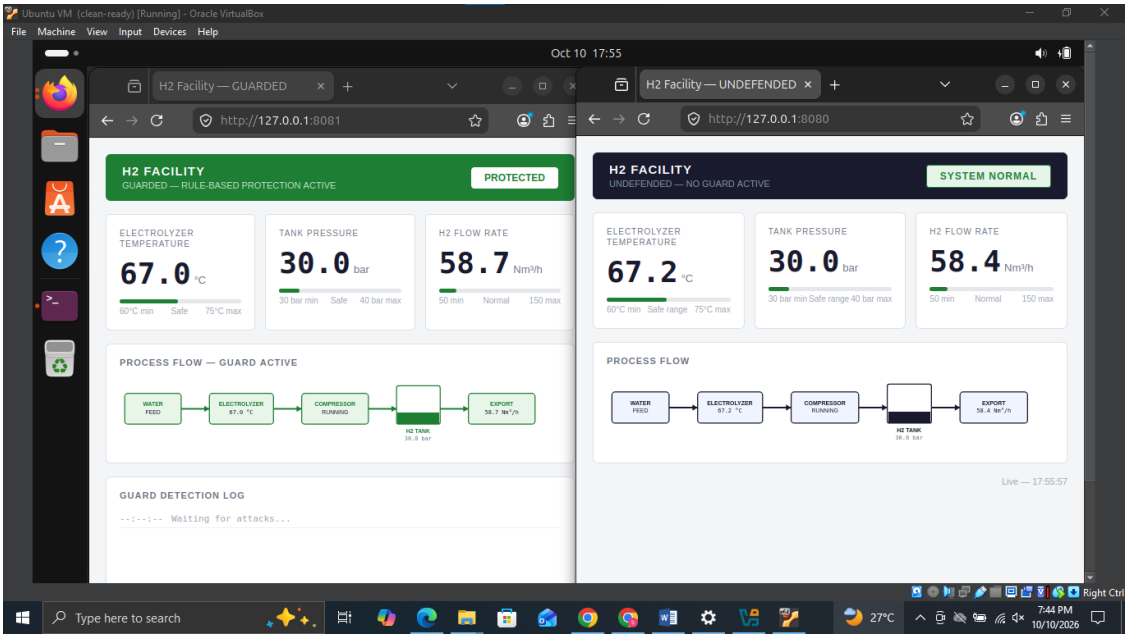
Direct dashboard (8080) — UNDEFENDED / SYSTEM NORMAL

Figure: Direct dashboard (8080) — UNDEFENDED / SYSTEM NORMAL



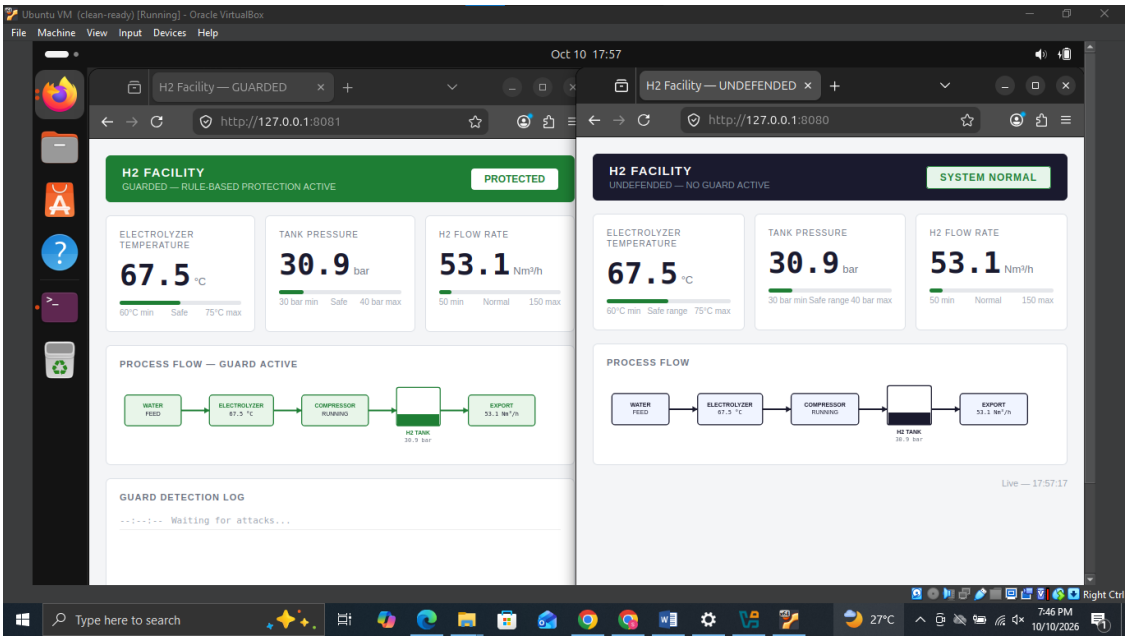
Guarded dashboard (8081) — PROTECTED, log waiting for attacks

Figure: Guarded dashboard (8081) — PROTECTED, log waiting for attacks



Baseline — both dashboards agree (sample 1)

Figure: Baseline — both dashboards agree (sample 1)



Baseline — both dashboards agree (sample 2)

Figure: Baseline — both dashboards agree (sample 2)

3. The Modbus/TCP data model

Reading through `plant.py` told me precisely how the process values are represented on the wire, which I needed before I could make sense of any packet capture. The server exposes four holding registers under unit (slave) ID 1, and because it is configured with `zero_mode=True` the addressing is zero-based. Every value is stored as an integer in tenths of its engineering unit, so a raw register value of 705 means 70.5 °C, and the scaling has to be applied by hand when reading the numbers back.

Reg	Point	Scale	Normal bounds (raw → engineering)	Notes	Source
0	Electrolyzer temperature	÷10 → °C	600–750 → 60.0–75.0 °C	—	`plant.py:16,21`
1	Tank pressure	÷10 → bar	300–400 → 30.0–40.0 bar	drives the alarm	`plant.py:17,22`
2	H_2 flow rate	÷10 → Nm^3/h	500–1500 → 50.0–150.0	—	`plant.py:18,23`
3	Alarm	0 / 1	0 or 1	set to 1 at pressure ≥ 39.5 bar	`plant.py:19,79`

While mapping this out I noticed something worth flagging honestly, because the brief specifically asks for any mismatch between the source and what is displayed. The plant sets its own alarm register (register 3) to 1 as soon as pressure reaches 39.5 bar, but the dashboards completely ignore that register and compute their own alarm instead, only reacting when pressure climbs above 40.0 bar or falls below 28.0. The practical consequence is that the plant can be asserting its alarm somewhere between 39.5 and 40.0 bar while the dashboard still reassures the operator that all is normal. It is not a protocol fault, but it is a genuine monitoring blind spot.

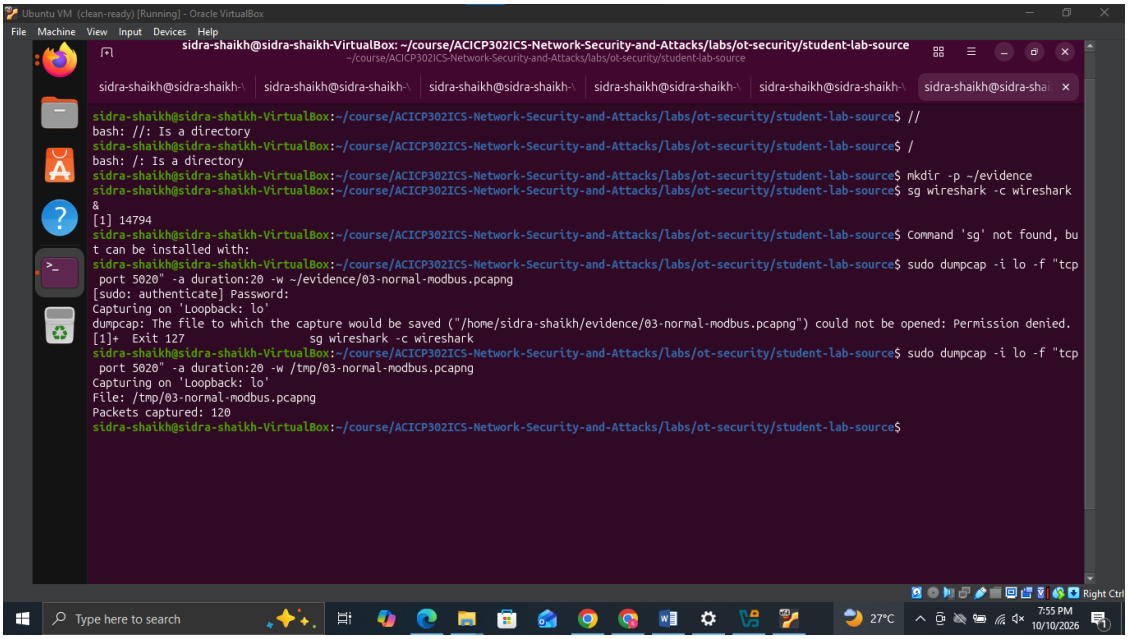
4. Packet visibility and protocol analysis

To see the protocol in motion I captured twenty seconds of ordinary traffic on the loopback interface with `dumpcap -i lo -f "tcp port 5020"`, which gave me 120 packets, every one of them travelling from `127.0.0.1` to `127.0.0.1`. Because the plant uses port 5020 rather than Modbus's usual 502, Wireshark did not recognise the protocol at first, so I used Analyze → Decode As to tell it that TCP port 5020 should be read as Modbus/TCP. After that the traffic decoded cleanly into queries and responses.

I then picked one request and its matching reply to work through in detail. The request came from a client on port 42076 and carried transaction identifier 2459, protocol identifier 0 (which simply means Modbus), unit identifier 1, and function code 3 — "Read Holding Registers" — asking for four registers starting at address 0. The response carried the same transaction identifier 2459, which is how I know it is genuinely the answer to that request, and returned eight bytes: registers reading 705, 379, 655 and 0. Applying the scaling, that is 70.5 °C, 37.9 bar, 65.5 Nm³/h and an alarm of 0 — all comfortably inside the safe ranges, so a perfectly normal reading.

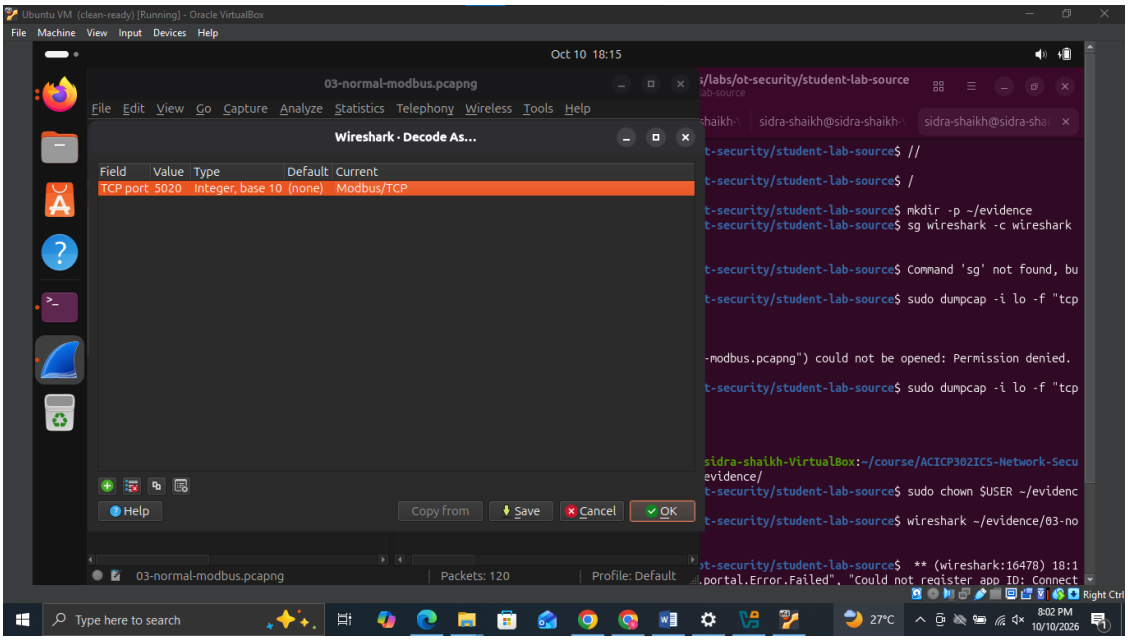
The nicest piece of corroboration came from lining the packet up against the plant's own log. The telemetry line the plant printed at 18:07:09 read `temp=70.5C pressure=37.9bar flow=65.5Nm3/h alarm=0`, which matches the decoded registers exactly. Wireshark even measured the round trip at about 1.29 milliseconds. I also noticed two separate transaction-ID streams interleaving in the capture, coming from ports 42076 and 57128 and each polling roughly once a second — these are the direct dashboard and the guard both reading the plant. Everything in the baseline was a function-3 read; there were no writes at all.

It is worth being clear about what this capture does and does not establish. It proves that the Modbus traffic stays on loopback, that normal activity is read-only, and that the register values match both the source scaling and the live process. What it cannot prove is *who* the client is, because Modbus/TCP carries no authentication whatsoever — any local process that can open the port can read or write freely. It also says nothing about the guarded path on 5021, which I did not capture, nor about behaviour under attack.



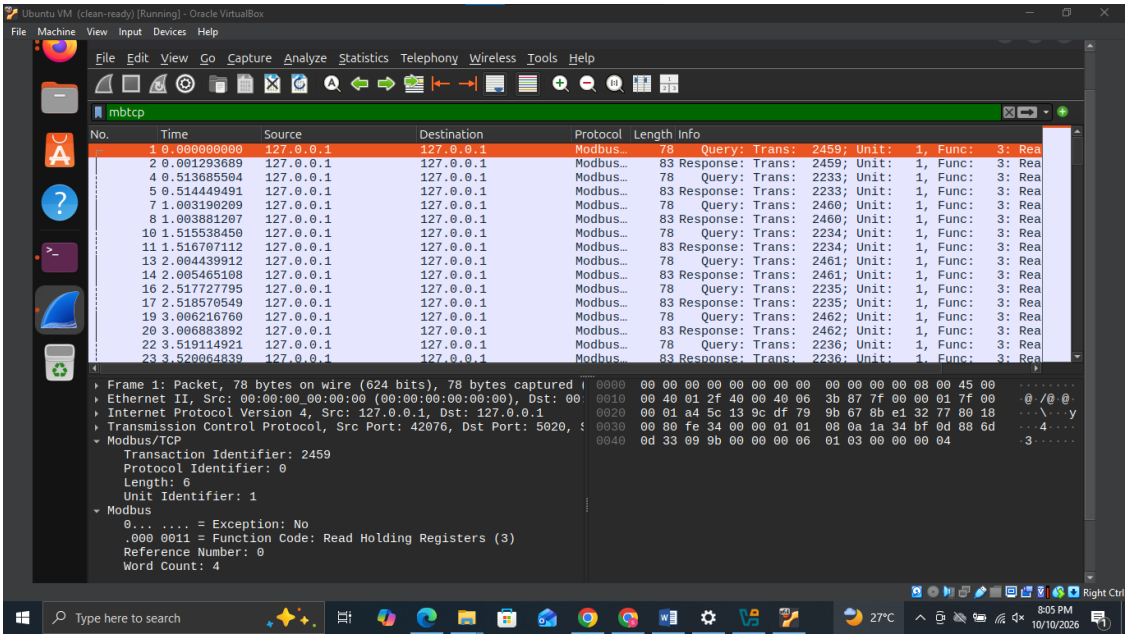
dumpcap captured 120 loopback packets on tcp port 5020

Figure: dumpcap captured 120 loopback packets on tcp port 5020



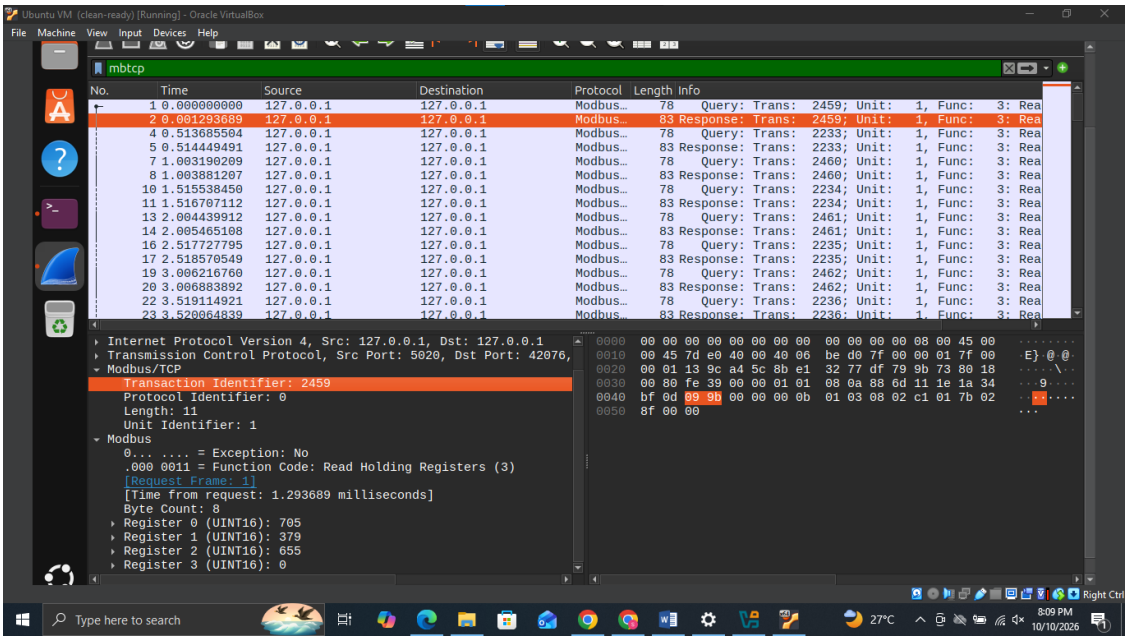
Decode As — TCP port 5020 read as Modbus/TCP

Figure: Decode As — TCP port 5020 read as Modbus/TCP



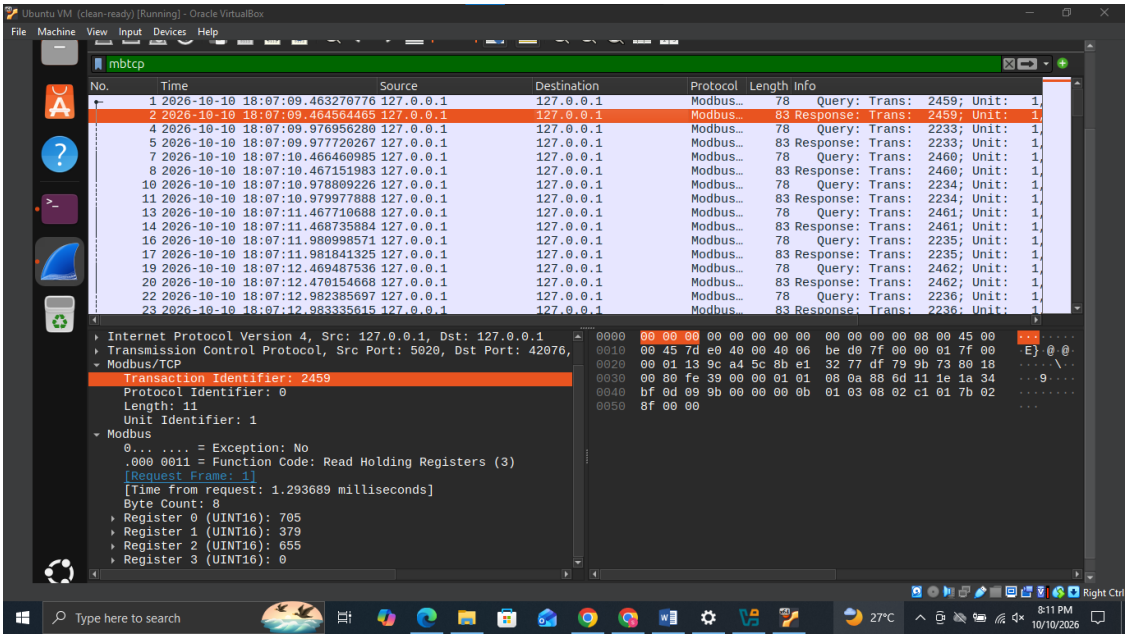
Request — transaction 2459, function 3, read 4 registers from 0

Figure: Request — transaction 2459, function 3, read 4 registers from 0



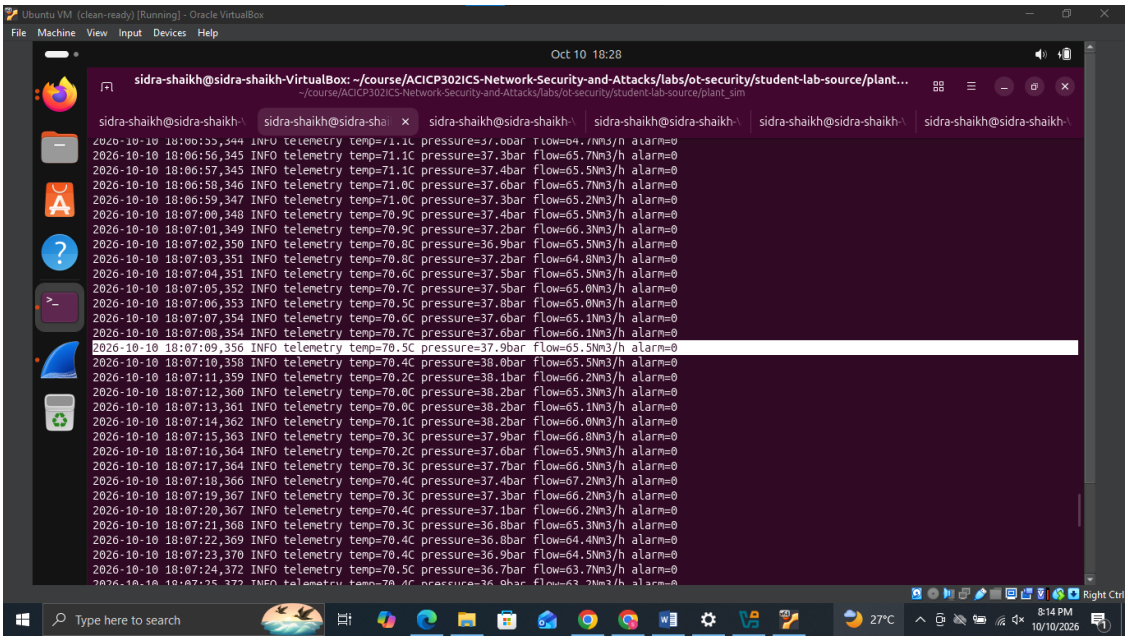
Response 2459 — 705/379/655/0 = 70.5°C / 37.9bar / 65.5 / alarm 0

Figure: Response 2459 — 705/379/655/0 = 70.5°C / 37.9bar / 65.5 / alarm 0



Response timestamped 18:07:09

Figure: Response timestamped 18:07:09

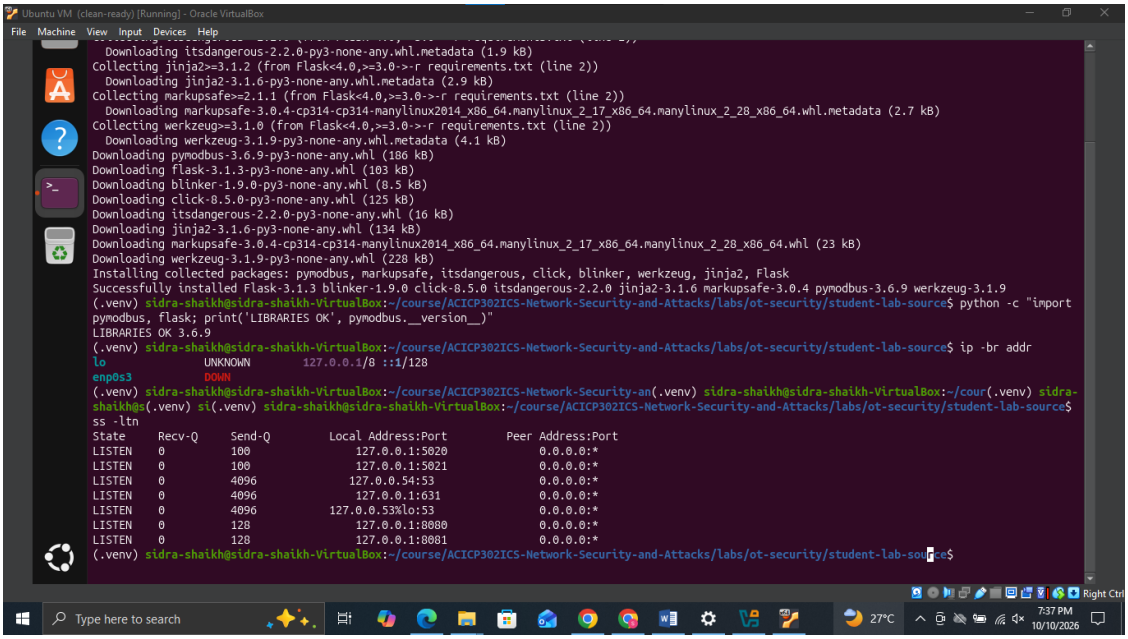


Plant log at 18:07:09 matches the decoded registers

Figure: Plant log at 18:07:09 matches the decoded registers

5. Safe discovery and exposure mapping

Keeping strictly within the loopback scope, I used `ss -ltn` together with source inspection to map what was actually listening, rather than scanning anything. The four lab services were all bound to `127.0.0.1` and nothing else — the plant on 5020, the guard on 5021, and the two dashboards on 8080 and 8081. The only other listeners were ordinary Ubuntu system services (systemd-resolved's DNS helper and the CUPS printing service), which are not part of the lab and are themselves loopback-only.



ss -ltn — all four lab ports bound to 127.0.0.1 only

Figure: ss -ltn — all four lab ports bound to 127.0.0.1 only

From the architecture and the behavior I observed, I drew up the following risk register. Each entry is tied to something I actually saw rather than something I assumed.

ID	Finding	Likelihood	Impact	Rating	Preventive control	Detective control	Guard covers?
R1	Modbus/TCP has no authentication; any local client can read or write (seen in the capture)	High	High	High	Per-client authentication / allow-listing; segmentation	Connection logging	No
R2	The direct path on port 5020 bypasses the guard completely (seen in T1a)	High	High	High	Make the plant reachable only through the guard; firewall 5020	Alert on direct-port connections	No
R3	The guarded dashboard can keep showing stale “safe” values while the plant is attacked directly, because the guard only mirrors in-range readings	Medium	High	High	Reconcile the guard’s mirror against the raw plant and flag divergence	Divergence alarm	Partial
R4	No replay or command-rate detection; repeated in-range writes are accepted (seen in T3)	Medium	Medium	Medium	Sequence numbers / nonces or write-rate limits	Rate-anomaly detection	No
R5	The alarm register is ignored by the dashboards, so the plant’s alarm and the displayed status can disagree	Medium	Medium	Medium	Display register 3 directly	Alarm-mismatch check	No

6. Analysing the supplied guard

The guard is written as a write-through firewall: a custom `GuardedContext` validates every incoming write before deciding whether to pass it down to the plant. The design is quite strict — if a write fails validation it is dropped entirely, nothing is forwarded, the guard’s own mirror

is left untouched, and the value is not even added to the history the drift check relies on, so the last known-safe value stays authoritative. It applies three checks, which I confirmed both by reading the code and by watching them fire during my demonstrations.

Check	Source	What it tests	Threshold (temp / pressure / flow, raw)	On failure
Absolute range	`guard.py:92–96`	value outside the physical safe band	600–750 / 300–400 / 500–1500	drop + log
Step	`guard.py:98–103`	single jump too large	30 / 50 / 100 (3.0 °C / 5.0 bar / 10.0 Nm^3/h)	drop + log
Rolling drift	`guard.py:105–112`	creep across the last 3 accepted writes	20 / 20 / 40 (2.0 °C / 2.0 bar / 4.0 Nm^3/h)	drop + log

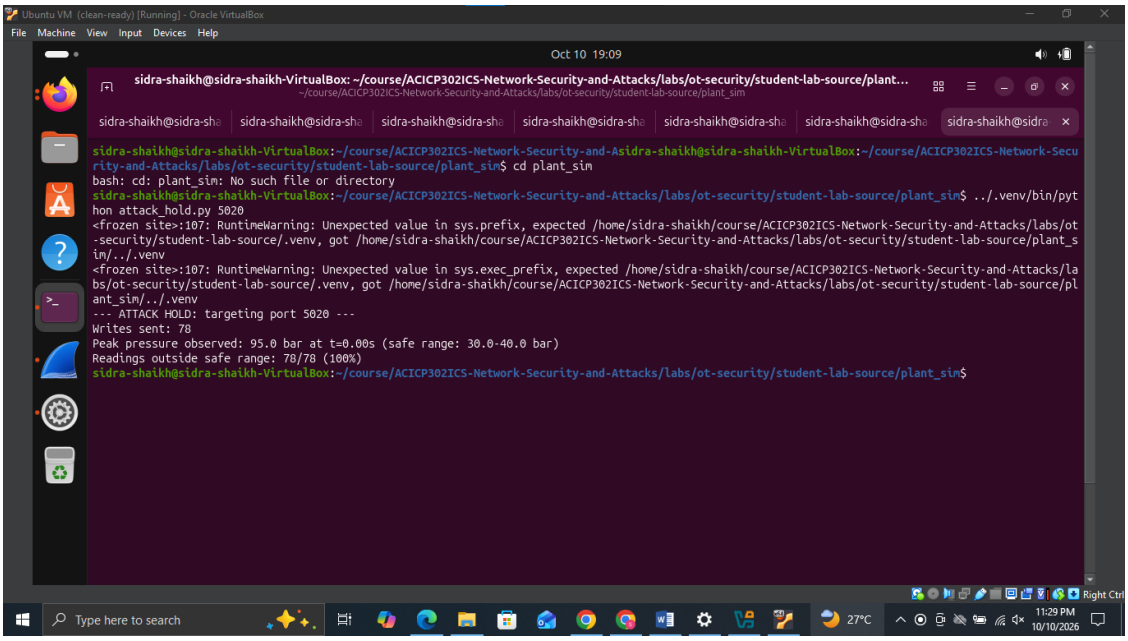
Every block is written as a line of JSON to `events.jsonl` and also surfaced in the guarded dashboard's detection log, which gave me clear audit evidence to capture. One of the blocked lines, for example, read: *"95.0 bar outside safe range 30.0-40.0 bar; step of 59.1 bar exceeds max 5.0 bar; cumulative drift of 59.8 bar over last 3 accepted writes exceeds 2.0 bar limit"* — all three checks objecting to the same write at once. There is also a background routine (`sync_safe_reads`) that mirrors the plant's live telemetry into the guard's displayed state, but only when every value is in range, which is what stops a direct-port attack from quietly bleeding corrupted numbers into the guarded view. That is a genuine strength for display integrity, though it is also exactly the mechanism behind risk R3.

It is important to be honest about what the guard does *not* do, as distinct from what one might wish it did. It has no concept of who a client is, so it cannot authenticate or authorise anyone. It only protects the path that goes through it, so the direct port 5020 bypasses it completely. It does nothing about availability or flooding. It cannot spot a replayed or duplicated command as long as the value itself is reasonable. And it never validates the alarm register. My recommendations — kept deliberately separate from what is actually implemented — would be to force all traffic through the guard and firewall off 5020, add per-client authentication, add replay and write-rate detection, reconcile the guarded mirror against the raw plant, and surface the real alarm register on the dashboards.

7. Controlled validation — three scenarios

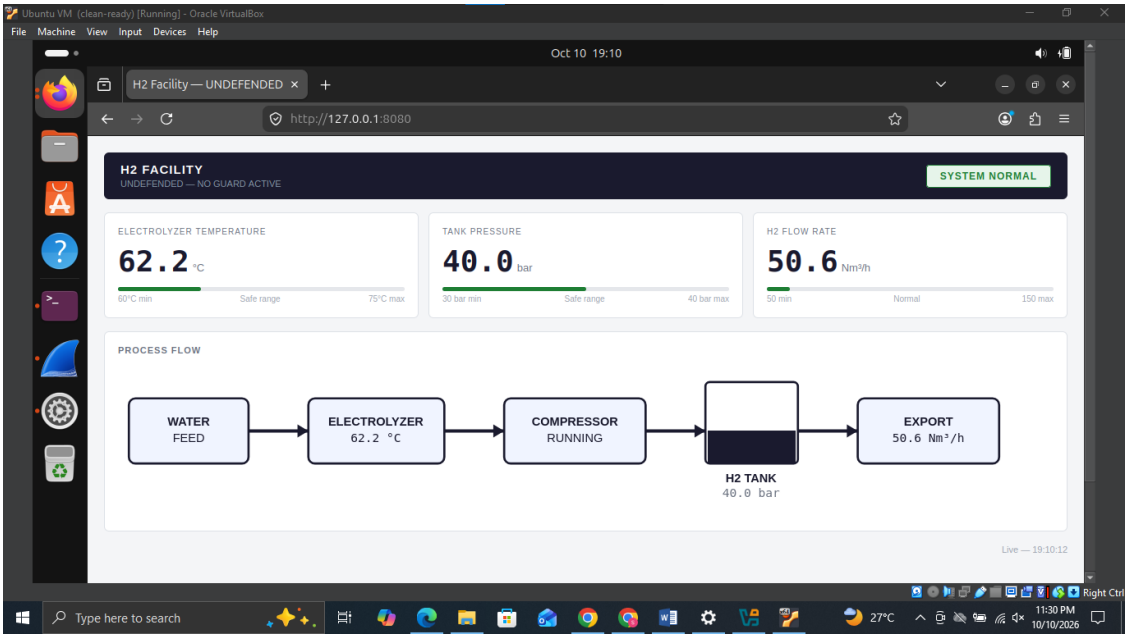
I ran three of the supplied demonstrations, each with its parameters untouched, and each one first against the direct port 5020 and then against the guarded port 5021 so that I could compare the two paths directly. After every run I checked that the values had returned to their normal range before moving on.

The first scenario, `attack_hold.py`, repeatedly writes a wildly out-of-range 95.0 bar for eight seconds. Against the direct port the result was stark: all 78 writes landed, every single reading was outside the safe range, and the script declared "UNSAFE VALUE REACHED THE PLANT". The only reason the dashboard still showed 40.0 bar afterwards is that the plant clamps its own pressure at that ceiling — there was no security control involved at all. Against the guarded port the same attack was, in the script's own words, "fully contained": all three guard checks fired on every write, nothing got through, and the pressure never left the safe range.



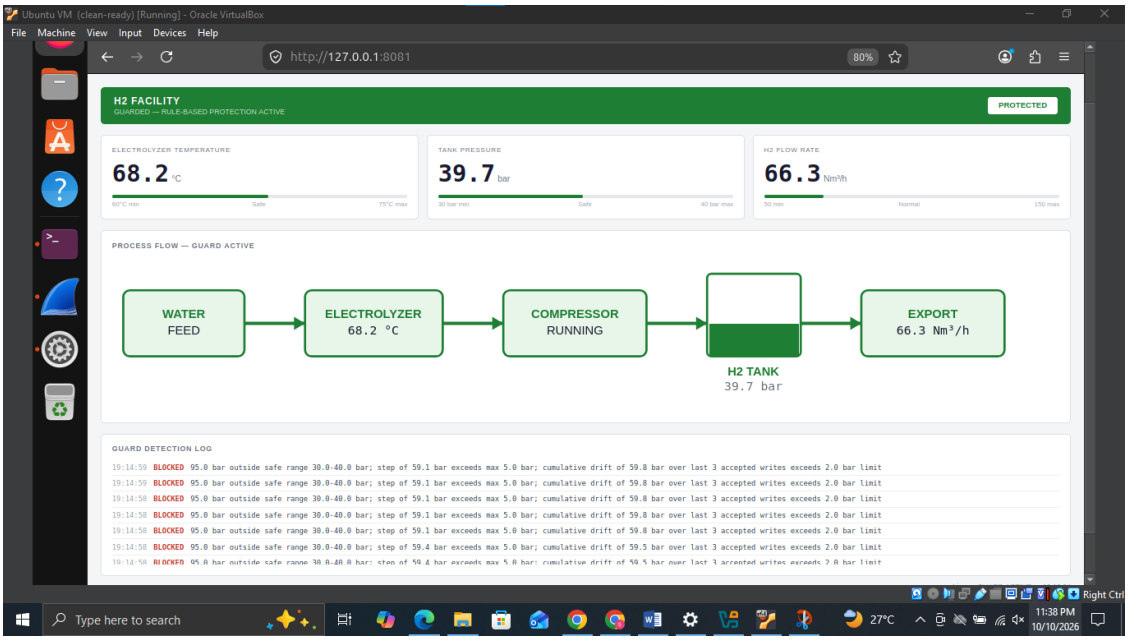
T1 outputs — 100% unsafe on :5020 vs fully contained on :5021

Figure: T1 outputs — 100% unsafe on :5020 vs fully contained on :5021



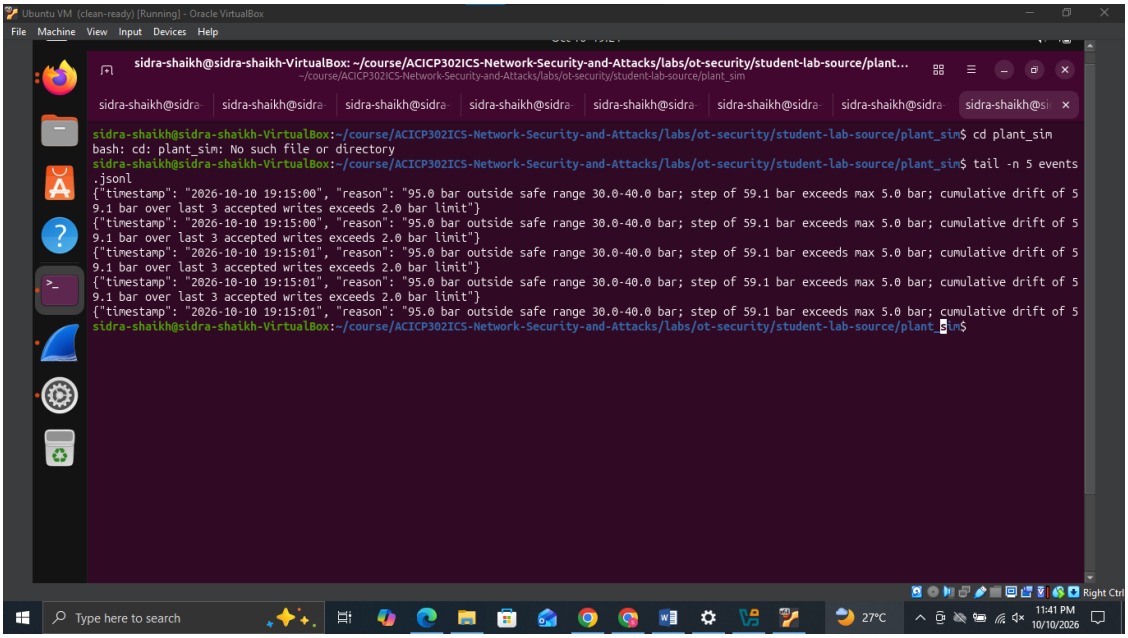
T1 direct path — dashboard clamped at 40.0 bar

Figure: T1 direct path — dashboard clamped at 40.0 bar



T1 guarded path — Guard Detection Log blocking every write

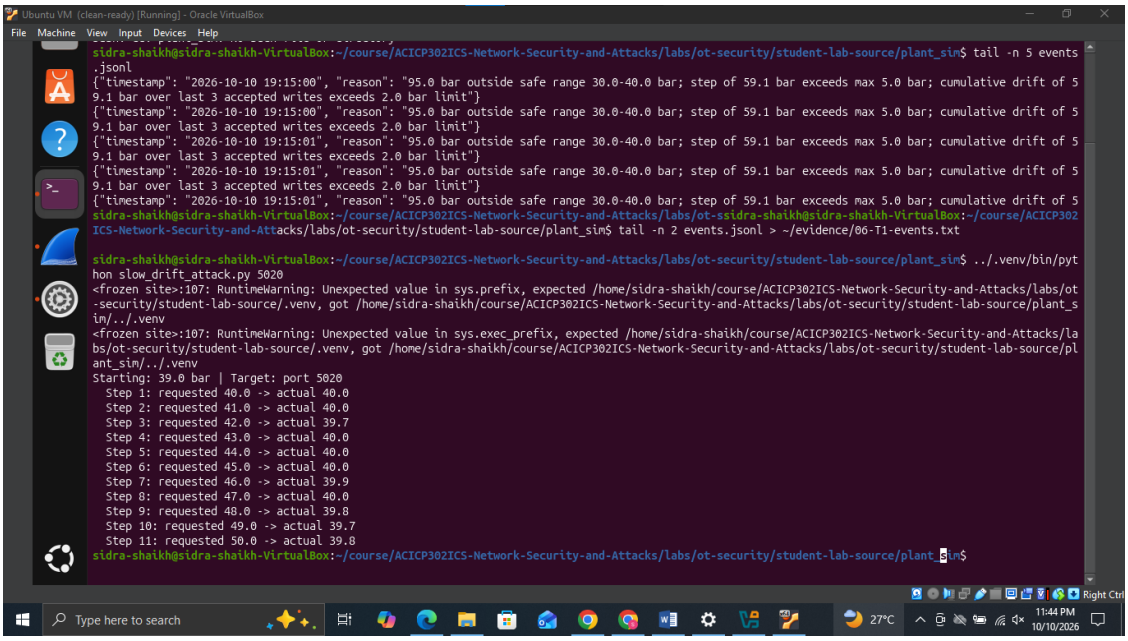
Figure: T1 guarded path — Guard Detection Log blocking every write



events.jsonl — JSON audit lines for the blocked writes

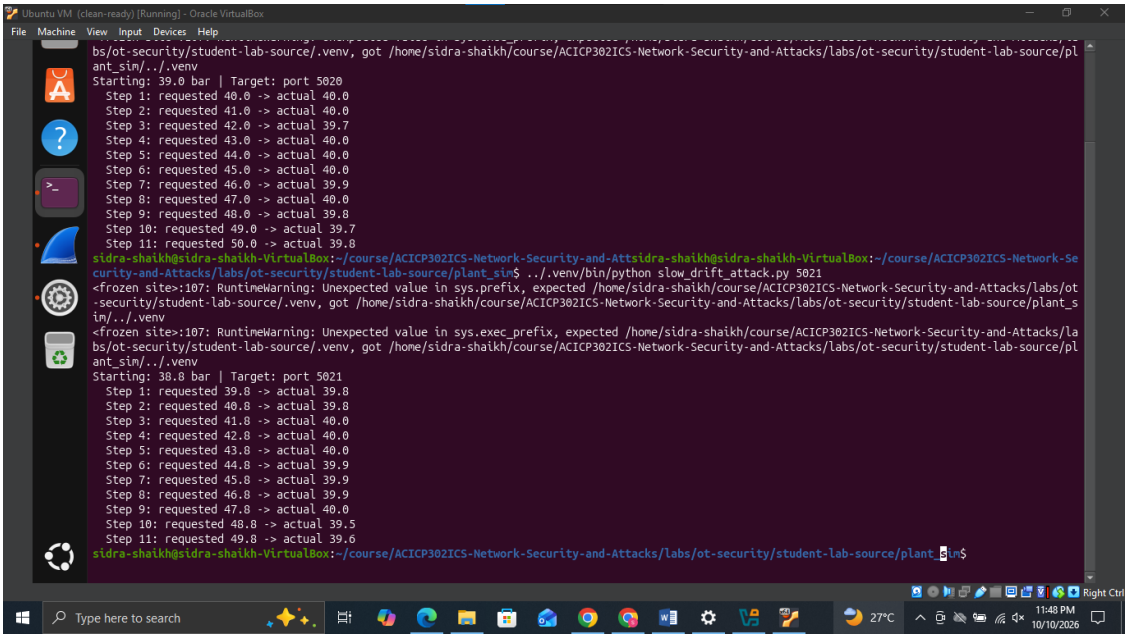
Figure: events.jsonl — JSON audit lines for the blocked writes

The second scenario, `slow_drift_attack.py`, is sneakier, nudging the pressure up by one bar every couple of seconds in the hope that no single step looks alarming. On the direct port the requested value climbed from 40 toward 50 bar, but the actual readings stayed pinned around 39.7–40.0 because of the plant's own clamp — again, physics rather than security doing the limiting. On the guarded port the very first small step (to 39.8 bar, still in range) was accepted, but every write after that was blocked. Because this particular script climbs straight past 40 bar, it was actually the range check that caught it first, with the step and drift checks piling on once the gap from the last accepted value grew — which is why the later log lines cite all three reasons together. I think it is worth stating plainly that the drift check would be the star of the show against an attack that stayed *within* the range, which this one does not.



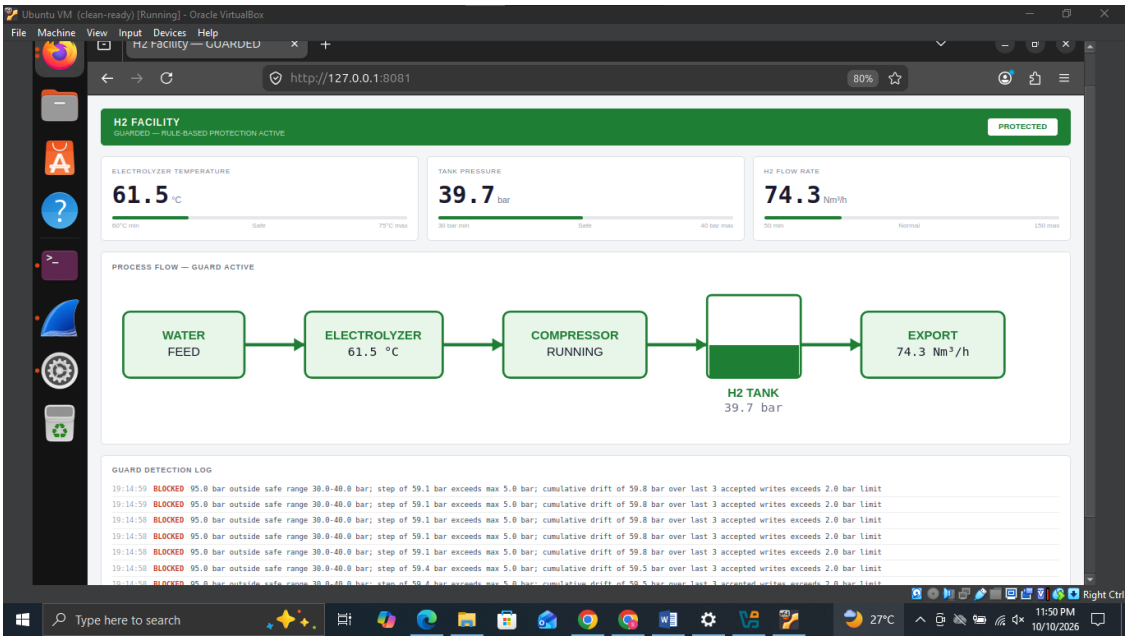
T2 direct path — drift clamped around 40 bar

Figure: T2 direct path — drift clamped around 40 bar



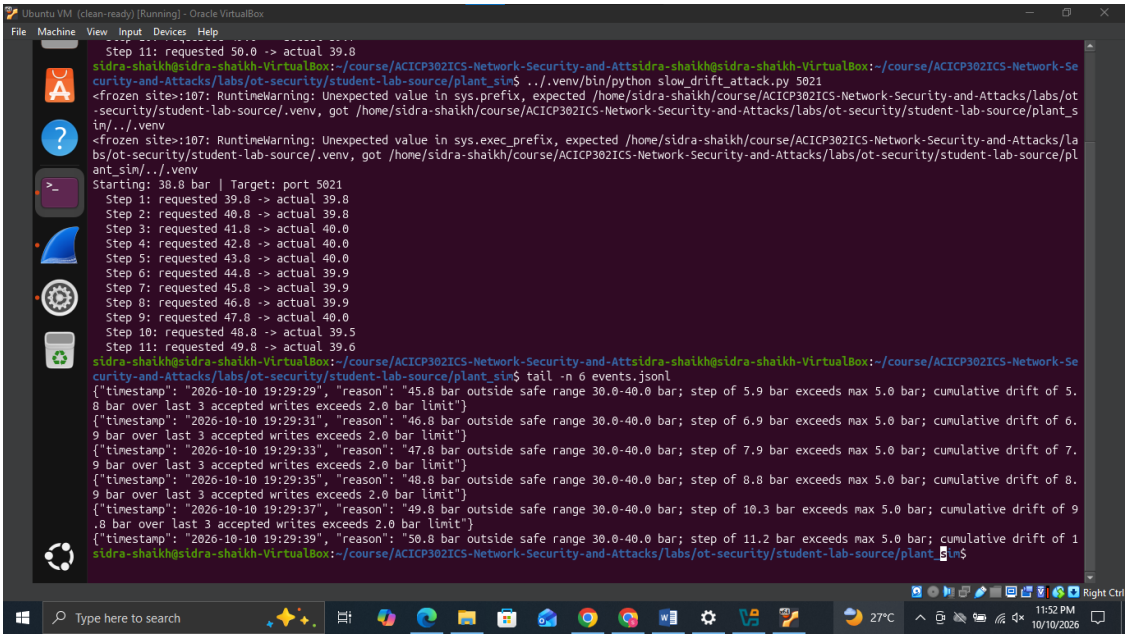
T2 guarded path — first step accepted, rest blocked

Figure: T2 guarded path — first step accepted, rest blocked



T2 guarded path held at ~39.7 bar

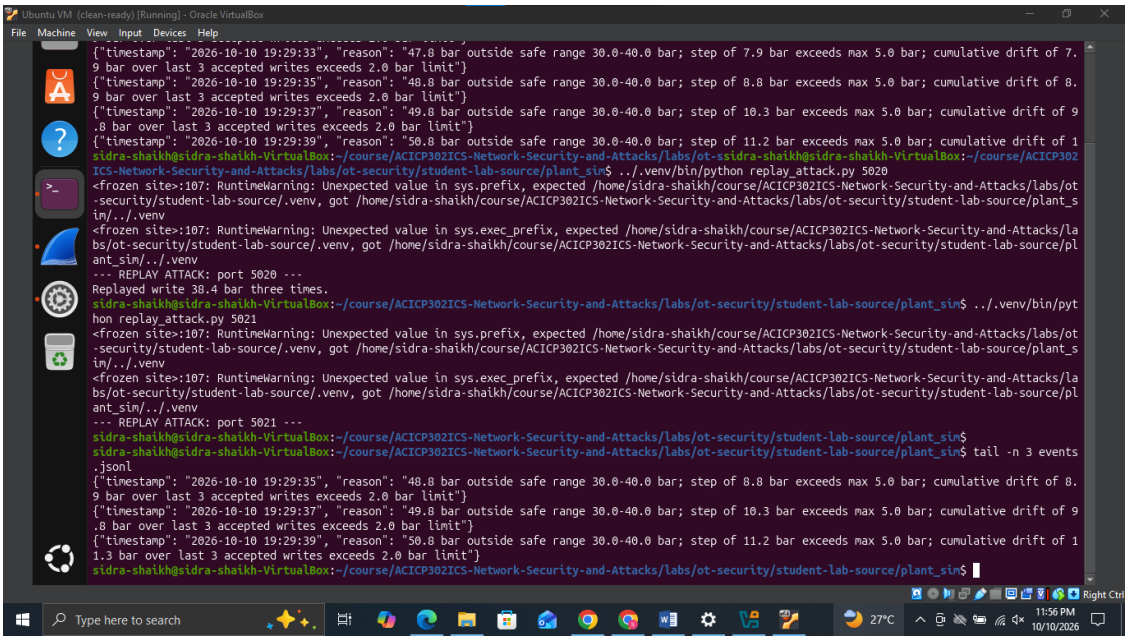
Figure: T2 guarded path held at ~39.7 bar



events.jsonl — T2 block reasons (range, step, drift)

Figure: events.jsonl — T2 block reasons (range, step, drift)

The third scenario, `replay_attack.py`, simply sends the same small, in-range write three times over. This one was accepted on both paths, and the guard logged nothing at all. That is precisely the point: because the value itself is perfectly reasonable, the guard — which only ever judges the value, never the sender or the history of commands — has no reason to object. It is the clearest demonstration in the whole lab of the guard's lack of authentication and replay detection, tying back to risks R1 and R4.



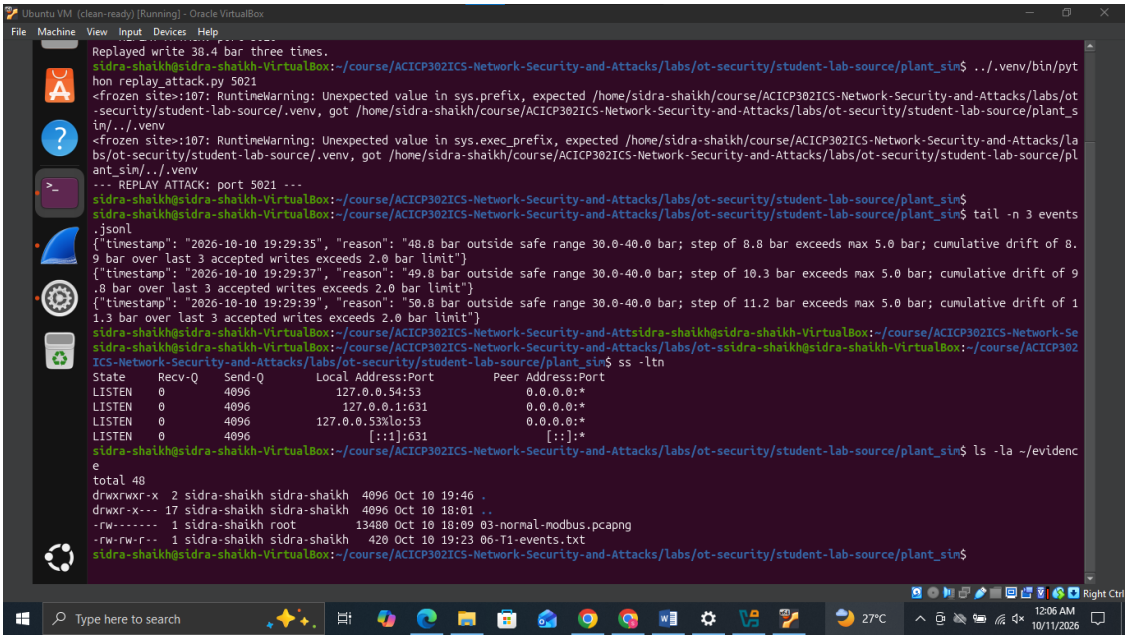
T3 replay — accepted on both paths, no guard events

Figure: T3 replay — accepted on both paths, no guard events

In each case the process settled back into its normal mid-range within about half a minute, and both dashboards returned to SYSTEM NORMAL and PROTECTED.

8. Recovery, cleanup and evidence

When I was finished I stopped all four services with Ctrl+C and confirmed the cleanup with `ss -ltn`, which showed that ports 5020, 5021, 8080 and 8081 were no longer listening — only the ordinary operating-system services remained. I kept the final guard log as `evidence/06-events-full.jsonl` and the packet capture as `evidence/pcap/03-normal-modbus.pcapng`. No credentials, keys or non-loopback captures were produced, and I did not copy the course source into my own repository — I reference it only by commit and file-and-line. The `clean-ready` snapshot remains available to restore if the instructor asks.



Cleanup — ss -ltn shows the lab ports no longer listening

Figure: Cleanup — ss -ltn shows the lab ports no longer listening

I should be candid about the limits of what I have shown. These results come from a single run on one virtual machine; I deliberately did not run the availability/flood demonstration (`dos_attack.py`), since the brief requires explicit instructor approval for it; and the thresholds and ranges throughout are those of the supplied simulation rather than any real plant.

9. Live defence plan

If asked to demonstrate this live, I would start by showing both dashboards sitting quietly at their normal baseline, with the guard log still waiting for attacks. I would then run the hold attack against the guarded port to show it being contained and logged, and immediately contrast that with the same attack against the direct port to show the value sailing straight through — the bypass that is risk R2. From there I would explain the single most important limitation, the absence of any authentication or replay detection, using the replay result. Finally I would stop everything and show a clean `ss -ltn` to prove the lab had been shut down properly.

10. References

The instructor-supplied source is the course repository at `github.com/icdfa/ACICP302ICS-Network-Security-and-Attacks`, used at commit `2a66f9bcf253eb3fe31c68ad52794cd535421da8` for reference only and not as my own work. I also relied on the Modbus Application Protocol specification (v1.1b3) for the meaning of function code 3, Read Holding Registers, and on the course's own Student Lab Guide.

Everything in this repository is my own analysis, captured inside my assigned isolated Ubuntu virtual machine on 127.0.0.1. I did not execute `archive-original/`, I used no bridged networking or port forwarding, and I left every demonstration's parameters unchanged.